Experiment 9

Clustering: K-Means Algorithm

Aim:

To perform clustering using the K-Means algorithm and determine the optimal number of clusters using the Elbow method.

Theory :

Clustering is the task of grouping observations in such a way that members of the same cluster are more similar to each other and members of different clusters are very different from each other. Clustering is commonly used to explore a dataset to either identify the underlying patterns in it or to create a group of characteristics.

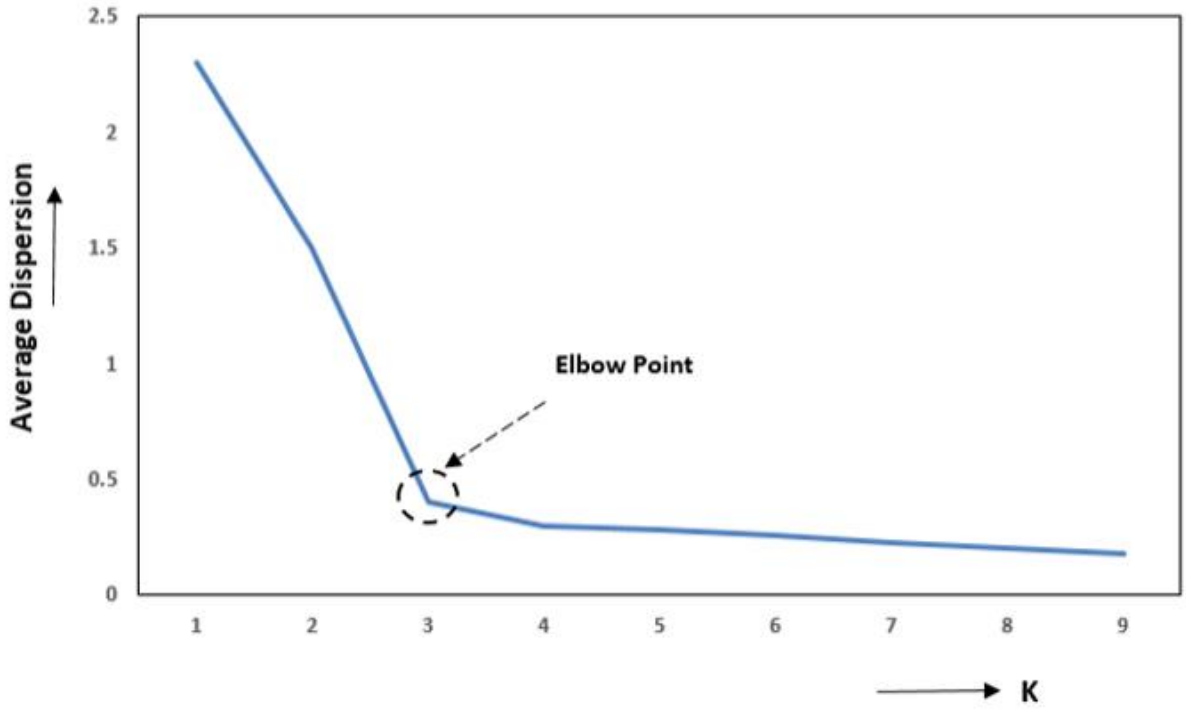
The k-means clustering algorithm is an iterative process of moving the centers of clusters or centroids to the mean position of their constituent points, and reassigning instances to their closest clusters iteratively until there is no significant change in the number of cluster centers possible or number of iterations reached.

The cost function of k-means is determined by the Euclidean distance (square-norm) between the observations belonging to that cluster with its respective centroid value. An intuitive way to understand the equation is, if there is only one cluster (k=1), then the distances between all the observations are compared with its single mean. Whereas, if, number of clusters increases to 2 (k= 2), then two-means are calculated and a few of the observations are assigned to cluster 1 and other observations are assigned to cluster two based on proximity. Subsequently, distances are calculated in cost functions by applying the same distance measure, but separately to their cluster centers:

Centroid distance calculations are performed by taking Euclidean distances. The Euclidean distance between two points A (X1, Y1) and B (X2, Y2) is shown as follows:

$$Euclidean\ distance\ between\ A\ \&\ B = \sqrt{(X2 - X1)^2 + (Y2 - Y1)\text{^}2}$$

To determine the optimal number of clusters, the Elbow method is used. The elbow method plots the value of the cost function produced by different values of k. The value of k at which improvement in distortion declines the most is called the elbow, at which we should stop dividing the data into further clusters. The following figure shows the Elbow method for selecting the optimal number of clusters. The elbow point indicates the most suitable number of clusters.



The elbow method is used to determine the optimal number of clusters in k-means clustering. The elbow method plots the value of the cost function produced by different values of k. The value of k at which improvement in distortion declines the most is called the elbow, at which we should stop dividing the data into further clusters.

Merits of K-means clustering:

- K-means is simple to understand and easy to implement, making it suitable for clustering large datasets.
- It is computationally efficient and scales well when the number of clusters and features is relatively small.
- The algorithm provides clear and compact cluster representations using centroids, which are easy to interpret.

Demerits of K-means clustering:

- It is sensitive to the initial selection of centroids and may converge to a local optimum.
- The algorithm performs poorly with non-spherical clusters, varying cluster sizes, and is highly sensitive to outliers.

Code:

Block 1: Importing Required Libraries

Input

```
import pandas as pd

import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
print("Libraries Imported");
```

Output

Libraries Imported

Block 2: Loading Dataset

Input:

```
data = pd.read_csv("Mall_Customers.csv")
Print("Dataset reading complete")
```

Output

Dataset reading complete

Block 3: Data Exploration - First Few Rows

Input:

```
data.head()
```

Output:

text

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Block 4: Data Exploration - Last Few Rows

Input:

```
data.tail()
```

Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

Block 5: Statistical Summary

Input:

data.describe()

Output:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

Block 6: Dataset Information

Input:

data.info()

Output:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 200 entries, 0 to 199

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64

```
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Block 7: Checking for Missing Values

Input:

```
data.isnull().sum()
```

Output:

```
CustomerID              0
Gender                  0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64
```

Block 8: Dataset Shape

Input:

```
data.shape
```

Output:

```
(200, 5)
```

Block 9: Encoding Categorical Variable

Input:

```
data['Gender'] = data['Gender'].map({'Male': 0, 'Female': 1})
data.head()
```

Output:

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1       0   19                  15                      39
```

1	2	0	21	15	81
2	3	1	20	16	6
3	4	1	23	16	77
4	5	1	31	17	40

Block 10: Feature Selection

Input:

```
X = data[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Feature selection complete");
```

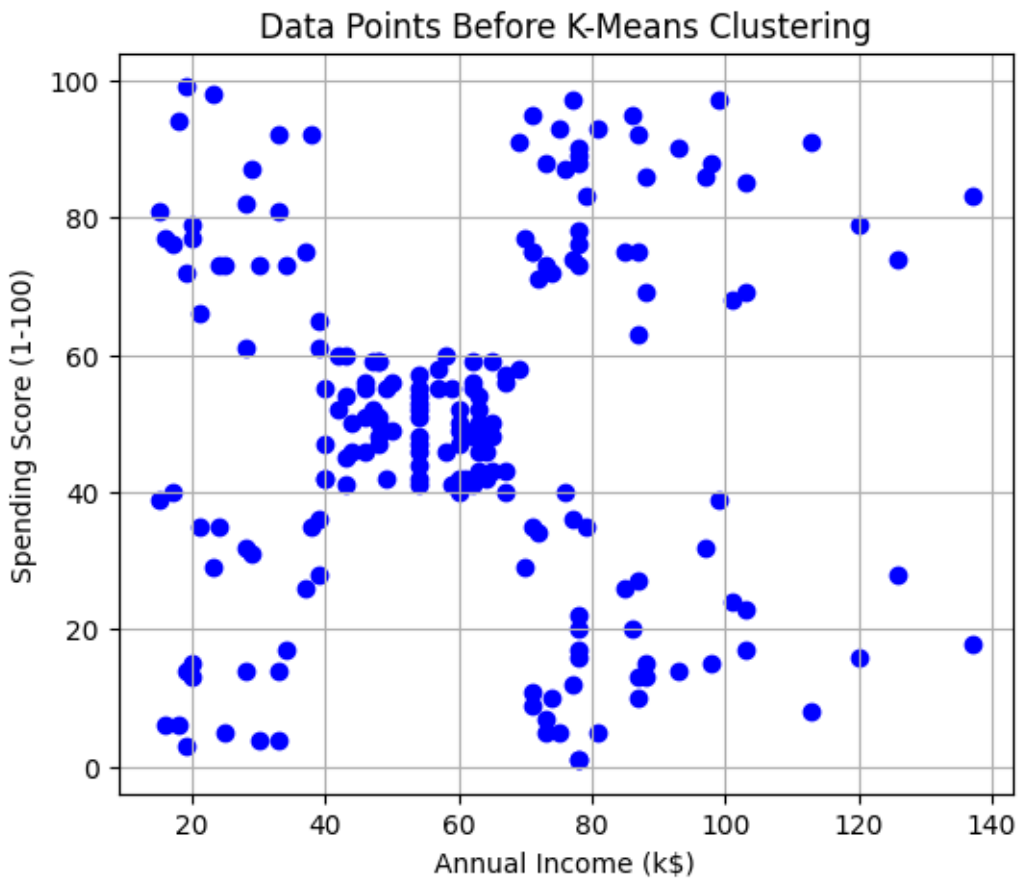
Output:
Feature selection complete

Block 11: Visualizing Data Points Before Clustering

Input:

```
plt.figure(figsize=(6,5))
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)'],
    color='blue'
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Data Points Before K-Means Clustering")
plt.grid()
plt.show()
```

Output:



Block 12: Feature Scaling

Input:

python

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("feature scaling complete")

Output:
feature scaling complete

Block 13: Finding Optimal K Using Elbow Method

Input:

wcss = []

for k in range(1,11):

    kmeans = KMeans(n_clusters=k, random_state=42)

kmeans.fit(X_scaled) # Random centroid initialization, Assignment of points to nearest centroid, Centroid update, Repeats until convergence

wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss, marker='o')

plt.xlabel("Number of Clusters (K)")

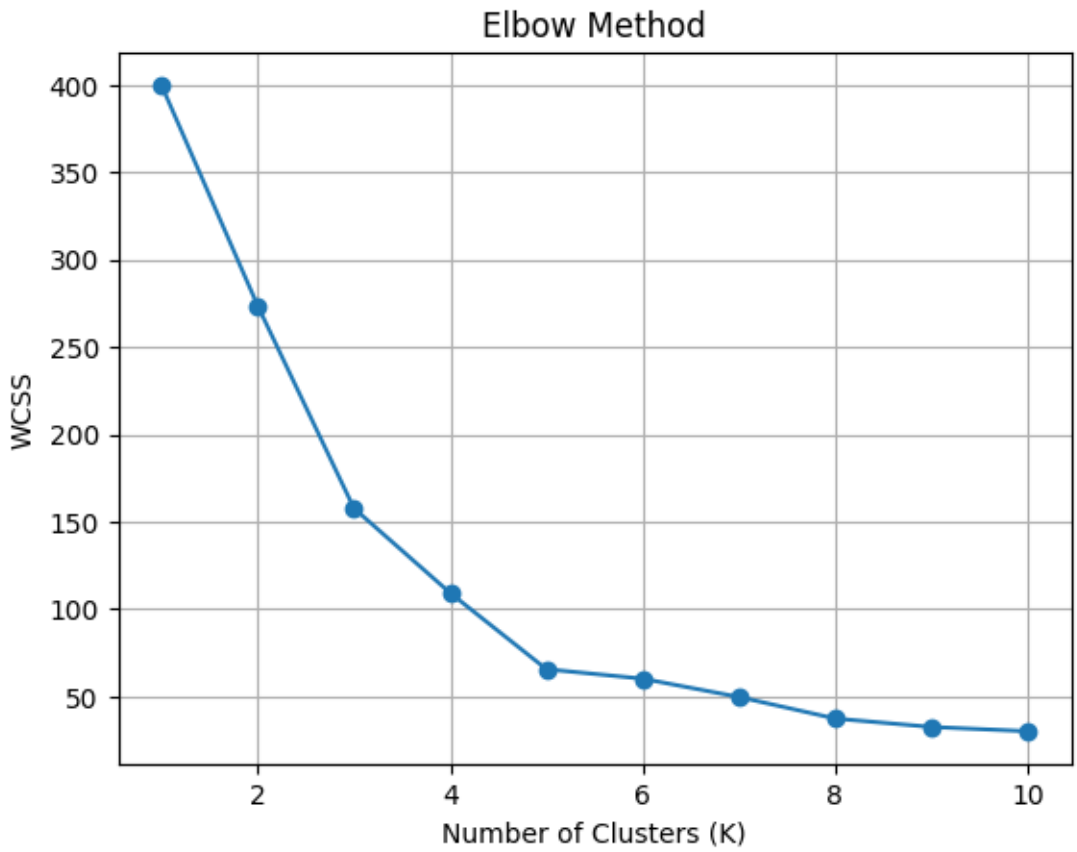
plt.ylabel("WCSS")

plt.title("Elbow Method")

plt.grid()

plt.show()

Output



Block 14: Model Training

Input

5. Model training

kmeans = KMeans(n_clusters=5, random_state=42)

kmeans.fit(X_scaled)

Output

```
KMeans
KMeans(n_clusters=5, random_state=42)
```

Block 15: Cluster

Input

```
labels = kmeans.labels_
data['Cluster'] = labels
data[['Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']].head()
```

Output

Block 16: Cluster Visualization

Input

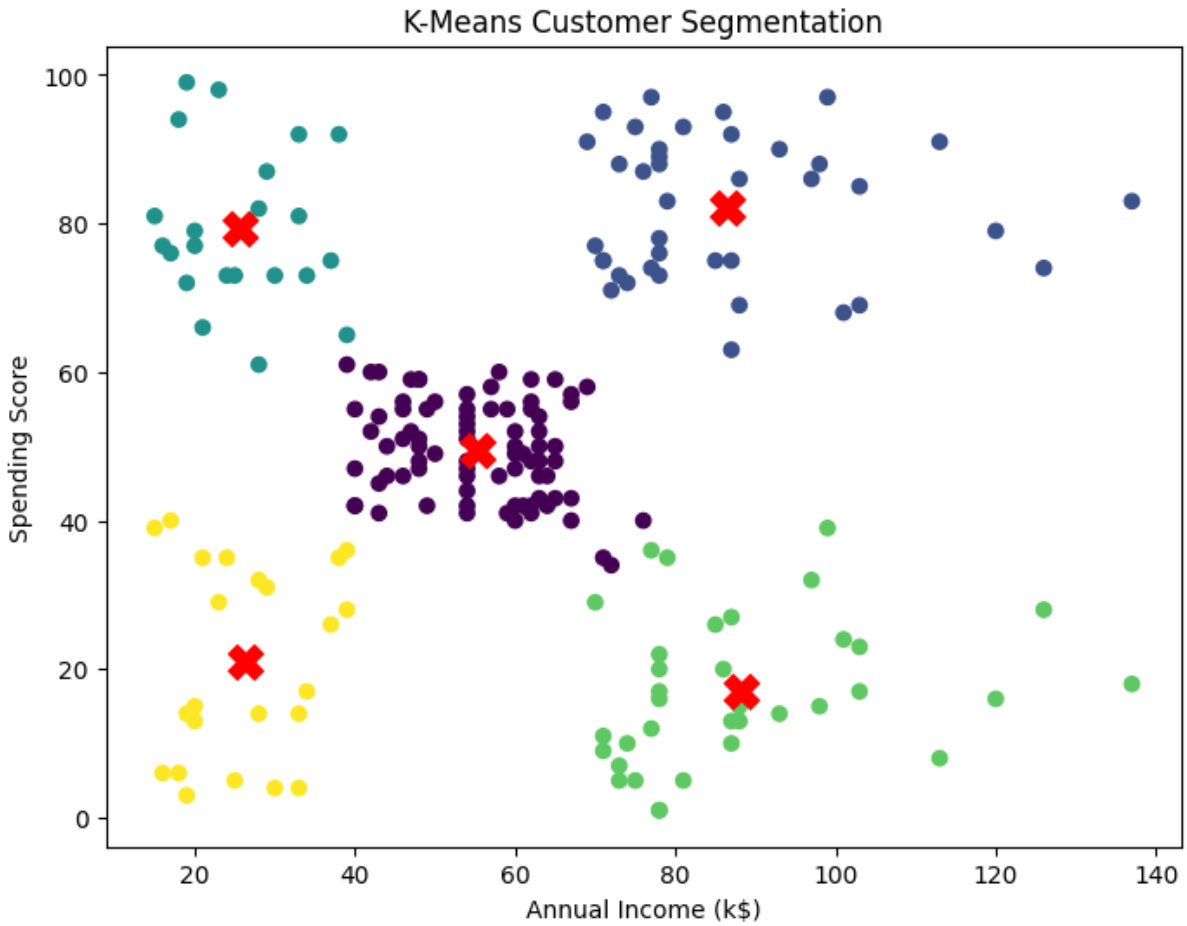
 # 7. Model Evaluation

```
plt.figure(figsize=(8,6))
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)'],
    c=labels,
    cmap='viridis'
)

centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centers[:,0],
    centers[:,1],
    c='red',
    s=200,
    marker='X'
```

```
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.title("K-Means Customer Segmentation")
plt.show()
```

Output



Block 17: Silhouette Score

Input

```
silhouette = silhouette_score(X_scaled, labels)
print("Silhouette Score:", silhouette)
```

Output

Silhouette Score: 0.5546571631111091

Block 18: wcss

Input

```
wcss = kmeans.inertia_

print("WCSS (Inertia):", wcss)
```

Output

```
WCSS (Inertia): 65.56840815571681
```

Block 19: Davies-Bouldin Index Evaluation

Input

```
db_score = davies_bouldin_score(X_scaled, labels)

print("Davies-Bouldin Index:", db_score)
```

Output

```
Davies-Bouldin Index: 0.5722356162263352
```